

Product Requirements Document

Image Scan (Liveness / KYC) & Biometric Authentication

App: BrowserStack Bank (Demo App) Version: 1.0 Date: 2025 Status: Draft

1. Overview

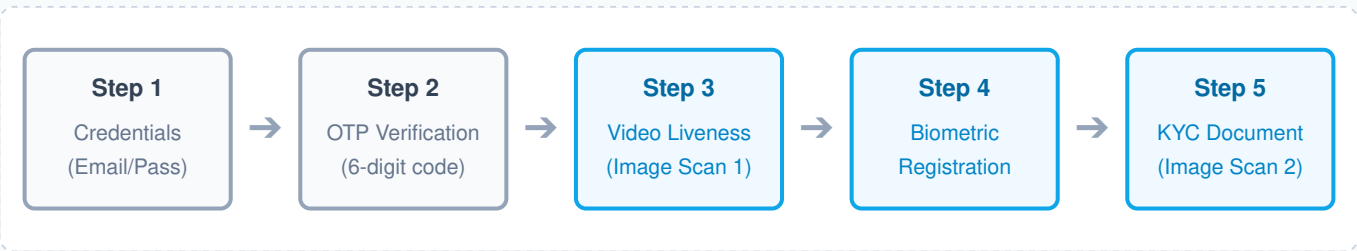
This PRD defines the requirements for two major security verification features within the BrowserStack Bank application. These features are designed to establish trust, ensure compliance, and streamline user access:

- **Image Scan** — A robust, two-part identity verification workflow active during new user signup. It comprises a **Video Liveness Check** (Step 3 of 5) to prove physical presence, followed by a **KYC Document Upload** (Step 5 of 5) to satisfy regulatory identity verification.
- **Biometric Authentication** — A native hardware-backed security gate utilizing device fingerprint or Face ID sensors. It acts as Step 4 of 5 during initial registration, and serves as a frictionless, standalone security checkpoint during subsequent logins.

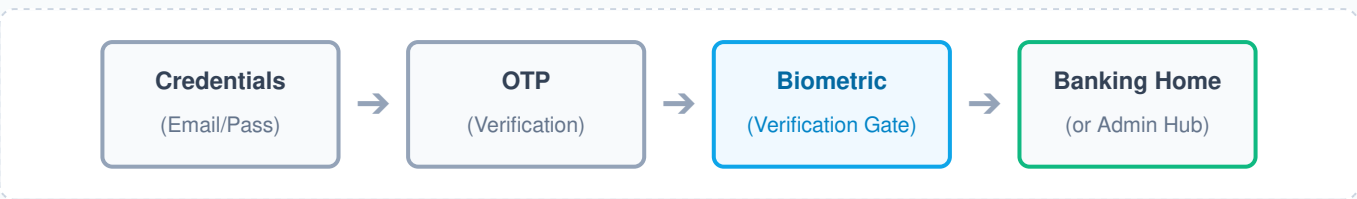
Both systems work in tandem to ensure a secure, verifiable, yet user-friendly onboarding and authentication lifecycle.

2. User Flows

2.1 Signup Onboarding (5-Step Flow)



2.2 Login Flow



2.3 Flow Config Gates

The onboarding path dynamically adapts based on the configuration state provided by `AuthStore.getFlowConfig()`.

Config Flag	Target Feature	Behavior when false
<code>cameraInjection</code>	Liveness Screen (Step 3)	The app completely skips the Liveness check and routes directly to <code>/biometric</code> .
<code>fileUpload</code>	KYC Screen (Step 5)	The app automatically marks KYC as complete in the backend and routes directly to <code>/(banking)/home</code> .

3. Feature: Video Liveness Check (Image Scan — Part 1)

3.1 Purpose

To verify that the individual signing up is a live, physically present human (anti-spoofing). This is achieved by recording an 8-second video via the front-facing camera and continually detecting eye presence using `expo-face-detector`.

3.2 Screen Details: `/liveness`

The screen operates through distinct phases to guide the user from preparation to successful verification.

Phase	UI State & Descriptions
ready	Live camera preview is rendered. The "Start Verification" action button is active.
recording	An 8-second visual countdown begins. The system takes facial snapshots every 800ms.
done	Liveness successfully verified. A "Continue to Biometric Setup" button is displayed.
retry	Triggered if eyes are not detected for >2 consecutive seconds. Displays an error state with a "Try Again" option.

Face & Eye Detection Logic

Snapshots are captured every **800ms** using `CameraView.takePictureAsync`. The `expo-face-detector` evaluates both `leftEyeOpenProbability` and `rightEyeOpenProbability` against a strictly **> 0.3** threshold.

- **Success Indicator:** An "Eyes detected ✓" badge overlays the camera frame when eyes are confirmed. Recording label updates to: `✓ Eyes detected · Xs remaining`.
- **Failure State:** If no eyes are detected for a continuous **2-second window**, the system aborts recording and transitions to the `retry` phase.
- **Fallback:** In environments lacking native face detector support, a successful camera snapshot alone implies user presence.

Permissions & Graceful Exit

Camera permissions are requested just-in-time when "Start Verification" is tapped. If denied, a dedicated "Camera Access Required" screen presents options to "Grant Camera Access" or "Skip this step". **Skip Option:** A skip action is globally available (on permission-denied, recording, and retry screens) which immediately routes the user to `/biometric`.

3.3 Acceptance Criteria

- ✓ Camera preview renders immediately when permission is granted.
- ✓ Countdown decrements accurately from 8 to 0 during the recording phase.
- ✓ "Eyes detected ✓" badge appears promptly when a face is centered in the frame.
- ✓ After 8 uninterrupted seconds with eyes detected, the app transitions to the `done` phase displaying "Video Verified Successfully".
- ✓ If eyes are lost/obstructed for 2 seconds, the app transitions to the `retry` phase displaying the "Eyes Not Detected" error.
- ✓ Tapping "Try Again" successfully resets the UI back to the `ready` phase.
- ✓ Tapping "Skip this step" strictly navigates to `/biometric` from any active phase.
- ✓ Denying camera permissions summons the permission-request error screen.
- ✓ The overarching Step badge explicitly reads: "Step 3 of 5 — Liveness Check".

3.4 Automation testIDs

Camera view: <code>liveness-camera-view</code>	Start Verification btn: <code>start-verification-btn</code>
Continue btn (done): <code>continue-btn</code>	Retry btn: <code>retry-btn</code>
Skip btn: <code>skip-liveness-btn</code>	

4. Feature: Biometric Authentication (Login Gate & Step 4)

4.1 Purpose

Leverage the operating system's native biometric capabilities (Fingerprint, Touch ID, Face ID) to authenticate the user securely. *Critical Privacy Note: No biometric data is extracted or transmitted to backend servers; all verification is processed strictly on-device.*

4.2 Modes & States: `/biometric`

Mode	Trigger Condition	Subtitle Display Text
Signup	<code>AuthStore.getFlow() === 'signup'</code>	"Register your fingerprint to secure your account"
Login	All other application flows	"Confirm your identity to access your account"

UI States:

- **idle:** Displays fingerprint icon; prompt reads "Tap to scan fingerprint".
- **scanning:** Activity spinner replaces icon; interaction buttons are disabled.
- **success:** Displays a green checkmark with "Identity Verified!"; automatically navigates forward after a 700ms delay.
- **error:** Displays a red X icon with a corresponding error message and a "Try Again" action button.

Authentication Logic & Routing

Utilizes `expo-local-authentication.authenticateAsync` customized with `promptMessage: "Verify your identity to continue"` and passcode fallback enabled (`disableDeviceFallback: false`).

Post-Success Navigation:

- Signup Flow + `fileUpload` enabled → Routes to `/kyc`
- Signup Flow + `fileUpload` disabled → Routes to `/(banking)/home`
- Login Flow + Admin Role → Routes to `/(admin)/users`
- Login Flow + Regular User → Routes to `/(banking)/home`

4.3 Acceptance Criteria

- ✓ Tapping the fingerprint circle or "Scan Fingerprint" invokes the OS-level native biometric prompt.
- ✓ Activity spinner is clearly visible during the scanning state, and primary buttons are disabled to prevent double-submission.
- ✓ Upon successful scan, UI updates to a green checkmark and auto-navigates exactly after 700ms.
- ✓ Upon scan failure, UI displays a red X, an explicit error message, and a functional "Try Again" button.
- ✓ When in Signup mode, the step badge specifically reads "Step 4 of 5 — Biometric Registration".
- ✓ When in Login mode, the step badge is entirely hidden.
- ✓ Passcode fallback is permitted and accessible via the native device prompt.
- ✓ The privacy disclaimer "Secured by device biometrics — no data leaves your device" is continually visible on the screen.
- ✓ Users designated with admin roles are appropriately routed to `/(admin)/users` post-login success.

4.4 Automation testIDs

BrowserStack logo: <code>bs-logo-bio</code>	Fingerprint circle: <code>fingerprint-btn</code>
Scan button: <code>scan-btn</code>	Error message: <code>bio-error</code>

5. Feature: KYC Document Upload (Image Scan — Part 2)

5.1 Purpose

Fulfill internal Know Your Customer (KYC) regulatory compliance by securely collecting a digital copy of a government-issued identity document (e.g., Passport, Driver's License) as the final step of onboarding.

5.2 Screen Details: `/kyc`

Document & Platform Constraints

- Must be a Government-issued photo ID (Passport, Driver's License).
- Must be strictly in **PDF format**.
- Must be clearly legible, with a strict maximum file size of **10 MB**.

Platform Behaviors: To circumvent OS-specific bugs, iOS utilizes a broad `*/*` MIME filter followed by manual `.pdf` validation (avoiding the blank picker bug), whereas Android utilizes strict `application/pdf` constraints.

User Interactions:

- **Consent Modal:** Mandated before the document picker is invoked. The user must explicitly accept consent before file selection begins.
- **Upload States:**
 - *No file:* Shows a dashed upload area with a cloud icon and "Upload Identity PDF".
 - *File selected:* Displays the selected file's name, size, a success checkmark, and a download button.
 - *Error:* Renders red alert text directly below the upload bounding box.
- **Download Feature:** Users can download/save the selected file back to their device using `expo-file-system copyAsync` targeting the `documentDirectory`. Alerts notify the user of "Download Complete" or "Download Failed".

5.3 Completion & Routing

Tapping "Complete KYC" invokes the `api.markKyc()` backend service, registering the document, before routing the user to `/((banking))/home`. Alternatively, users may tap "Skip", which directly routes to the home dashboard without firing the `markKyc` endpoint.

5.4 Acceptance Criteria

- ✓ The overarching Step badge explicitly reads: "Step 5 of 5 — KYC Verification".
- ✓ The Document Requirements card comprehensively lists all 4 stated requirements.
- ✓ Tapping the upload area universally triggers the consent modal prior to any system file picker.

- ✓ Accepting consent successfully invokes the native device file picker.
- ✓ On iOS, selecting a non-PDF file results in an immediate on-screen validation error.
- ✓ The UI dynamically updates to display the selected file's name and size post-selection.
- ✓ The Download button successfully saves the file to local storage and displays a success alert.
- ✓ Attempting to "Complete KYC" without an attached file displays the "Please upload your identity document" error.
- ✓ "Complete KYC" with a valid file triggers `markKyc` and successfully navigates to home.
- ✓ "Skip" cleanly navigates to home without attempting to call `markKyc`.
- ✓ If `fileUpload` config is set to false, the entire KYC screen is automatically bypassed.

5.5 Automation testIDs

Upload button: `upload-btn`

6. End-to-End Signup Architecture

The comprehensive navigation chain mapping all 5 steps combined under active configurations:

```
/index (login screen)
  → tap "Sign Up"
  → /signup (credentials)
  → /otp (6-digit OTP)
  → /liveness (Step 3 – camera liveness, if cameraInjection=true)
  → /biometric (Step 4 – fingerprint registration)
  → /kyc (Step 5 – PDF upload, if fileUpload=true)
  → /(banking)/home
```

7. Non-Functional Requirements

Category	Specification
Privacy	Biometric data is processed on-device exclusively; zero server transmission occurs.
Camera	Front-facing hardware is required for liveness; UI gracefully degrades if absent.
Permissions	Camera permission is requested strictly at runtime; denial states are handled gracefully with skip options.
Storage	KYC PDF uploads are capped at a maximum of 10 MB per file.
Offline Mode	Biometric authentication fully functions offline (device-local); liveness detection and KYC processing require active device storage and connectivity.
Accessibility	All primary interactive elements are mandated to have <code>testID</code> props to support UI test automation.
Error Recovery	All verification screens provide explicit retry and skip pathways to prevent UX dead-ends.

8. Out of Scope

- Server-side liveness video analysis (the current architectural implementation relies entirely on client-side probability processing).
- Multi-document KYC ingestion (system supports only a single primary document per signup instance).
- Biometric re-enrollment natively within the app after the initial registration (handled via OS settings).
- Distinct UI differentiation between Face ID vs. Fingerprint (the native OS abstraction layer handles presentation).

9. Technical Dependencies

Dependency Module	Application Usage
<code>expo-local-authentication</code>	Drives the native biometric prompt (Fingerprint / Face ID).
<code>expo-camera</code> / <code>CameraView</code>	Hooks into the front-camera feed for the liveness video element.
<code>expo-face-detector</code>	Computes eye-open probability thresholds dynamically.
<code>expo-document-picker</code>	Triggers OS-native file selection exclusively for PDF KYC documents.
<code>expo-file-system</code>	Handles local device file operations, including saving KYC documents.
<code>AuthStore</code>	Maintains global state for flow configurations, user roles, and flow-type gating.
<code>api.markKyc()</code>	Backend mutation endpoint to register KYC completion.